Capitolul 6

Compilatoare si Asambloare

Introducere in Limbajul de Asamblare

Conf. dr. ing. inf. Marius ROGOBETE

Software pentru generarea de programe în limbaj maşină

- *Software pentru generarea de programe în limbaj maşină* sunt de regulă compilatoarele şi asambloarele.
- Un **compilator** este un program care transformă programul scris într-un limbaj de programare de nivel înalt în limbaj maşină. Compilatoarele pentru un limbaj de programare vor avea partea „din faţă” (cea care recunoaşte construcţiile din limbajul de programare de nivel înalt) identică, iar „partea din spate” (cea care creează codul maşină) diferită pentru fiecare tip de sistem de calcul. Se poate ca acelaşi program compilat cu compilatoare diferite pentru acelaşi sistem de calcul să producă cod diferit.
- În procesul de compilare al unui program, programul sursă, scris într-un limbaj de programare de nivel înalt, este transformat în cod în limbaj de asamblare iar mai apoi codul în limbaj de asamblare este transformat în cod maşină de către asamblor. Aceste traduceri au loc în faza de compilare respectiv de asamblare. Programul obiect care rezultă poate fi link-editat cu alte programe obiect în faza de linkeditare.
- Programul link-editat, stocat de regulă pe disc, este încărcat în memoria principală în faza de încărcare a programului și este executat de către procesor în faza de execuție (run-time).
- Pentru activitatea de programare sunt utile generatoarele de programe. Acestea transformă programul sursă într-un nou format. Din această categorie fac parte: macrogeneratorul, asamblorul (relocabil, absolut), compilatoarele, interpretoarele, editorul de legături, bibliotecarul, editoarele de texte etc.

Software pentru generarea de programe în limbaj maşină

- Macrogeneratorul analizează un text sursă conţinând descrieri într-un limbaj special și prezintă la ieşire un fişier cu text scris în limbaj de asamblare, care poate fi prelucrat ulterior de asamblorul relocabil sau cel absolut. De exemplu, folosind TASM (al firmei Borland) se pot asambla programe în format (cod) Intel. Pentru a putea avea portabilitate între sistemele de tip Microsoft şi Linux, pentru procesoare Intel și compatibile se poate utiliza NASM. Pentru testarea programelor scrise în limbajul MMIXAL, pentru procesorul MMIX, se poate utiliza un simulator MMIX.
- Programul sursă, scris de utilizator în limbaj de (macro)asamblare, este constituit dintr-un număr de linii. Fiecare linie conține o instrucțiune a limbajului de asamblare sau o directivă de macrogenerare. Rolul macrogeneratorului este de a expanda macroinstrucțiunile existente şi a rezolva, pe cât posibil, blocurile condiţionale.
- Macrogeneratorul recunoaşte şi analizează: directivele de control numeric (tipul bazei de numeraţie), directivele de terminare (.**END**), directivele de asamblare condiţională (.**IF**, .**IFF**, .**IFT** etc.), directivele de definire a macroinstrucţiunilor (.**MACRO**, .**ENDM**), directivele de control (mesaje de eroare), directivele de generare și substituire, directivele de apelare a macrodefiniţiilor dintr-o bibliotecă etc.
- Asamblorul relocabil transformă modulele sursă scrise în limbaj de asamblare într-un modul obiect relocabil, o tabelă de simboluri și un listing de asamblare. Asamblorul relocabil acceptă la intrare unul sau mai multe fişiere sursă în limbaj de asamblare obţinute folosind macrogeneratorul sau scrise de utilizator. Ieşirile asamblorului constau dintr-un fişier obiect (**.obj**, **.o, .mmo** etc.), un fişier de listare (**.lst**) și o tabelă de simboluri (**.map**).

Software pentru generarea de programe în limbaj maşină

- Asamblorul absolut transformă modulele scrise în limbaj de asamblare (ca ieşire a macrogeneratorului sau scrise de programatori) într-un program executabil (**.tsk, .cmd, .out**, etc.)
- Compilatoarele efectuează translatarea programului sursă în program obiect relocabil. Pentru ca un astfel de modul să devină program executabil este necesară editarea legăturilor. Funcţia principală a editorului de legături este de a transforma şi înlănţui modulele obiect relocabile rezultate în procesul de asamblare şi/sau compilare pentru a obţine un program executabil acceptabil de programul încărcător al sistemului de operare. În faza de editare a legăturilor pot fi incorporate și module obiect situate în biblioteci relocabile.

Un compilator este structurat în patru componente principale:

- analizor lexical,
- analizor sintactic şi semantic,
- generator de cod și
- optimizator.

Toate aceste componente gestionează un tabel de simboluri.

- **Analizorul lexical** realizează o primă translatare a textului programului sursă într-un şir de entităţi lexicale ce constituie reprezentări interne ale unor categorii sintactice precum: cuvinte cheie, identificatori, constante, delimitatori, operatori etc. Astfel, în fazele următoare se poate lucra cu simboluri de lungime fixă și cu un număr mai mic de categorii sintactice. Analizorul lexical va completa tabelul de simboluri.
- **Analizorul sintactic şi semantic** verifică corectitudinea sintactică a instrucțiunilor şi colectează atributele semantice ale categoriilor sintactice din program. Ieşirea analizorului sintactic şi semantic o constituie o reprezentare codificată a structurii sintactice și un set de tabele ce conțin atributele semantice ale diferitelor categorii sintactice (simboluri, constante etc.)
- **Generatorul de cod** va traduce ieşirea analizorului sintactic şi semantic în format binar relocabil sau în limbaj de asamblare.
- **Optimizatorul** are rolul de a prelucra ieşirea generatorului de cod cu scopul de a minimiza memoria necesară la execuție şi a elimina redundanţele din corpul programului. Unele compilatoare efectuează anumite optimizări înainte de generarea codului.

Limbaje de nivel înalt, limbaje de nivel scăzut

- O funcție importantă a compilatorului constă în detectarea erorilor din programul sursă şi corectarea sau acoperirea lor. Spre deosebire de compilatoare, interpretoarele efecuează și executarea programului odată cu traducerea programului sursă, furnizând la ieșire rezultatele programului.
- Mai precis, diferenţa faţă de un compilator este aceea că interpretorul nu produce un program obiect ce urmează a fi executat după interpreatare, ci chiar execută acest program.
- Din punct de vedere structural, un interpretor se aseamănă cu un compilator, dar forma intermediară obţinută nu e folosită la generarea de cod, ci pentru a uşura decodificarea instrucţiunii sursă în vederea executării ei. Este cazul interpretorului sistemului AUTOCAD care analizează linia de comandă și, dacă comandă este corectă, realizează funcția solicitată.
- Un editor de texte este un program on-line (răspunsul la comenzi este imediat), ce acceptă comenzi introduse de la terminal pentru a scrie şi/sau şterge caractere, linii sau grupuri de linii din programul sursă sau din orice fişier text.
- Editoarele de texte pot lucra în mod linie şi/sau mod ecran. Ele pun la dispoziţie comenzi privind deplasarea cursorului în text (pe linie, în cadrul unui bloc sau în cadrul întregului fişier), manipularea blocurilor de text (marcare, deplasare, copiere, ştergere), comenzi de formatare a ieşirii etc. Exemplificăm prin Edit (Ms-DOS, Windows), Notepad (Windows), vi sau emacs (UNIX) etc.

Cu toate că majoritatea programelor se scriu în limbaje de nivel înalt, programatorii pot scrie programe sau secvențe din unele programe, care trebuie să ruleze foarte repede, în asamblare. În plus, s-ar putea să nu existe compilatoare pentru anumite procesoare speciale sau compilatoarele să nu poată fi folosite pentru a efectua anumite operații speciale. În aceste cazuri, programatorul este nevoit să recurgă din nou la limbajul de asamblare.

Limbajele de programare de nivel înalt ne permit să ignorăm arhitectura sistemului de calcul țintă. Pe de altă parte, la nivel limbaj maşină, arhitectura este cel mai important aspect de care trebuie ținut cont.

Software pentru generarea de programe în limbaj maşină

- Limbajele de programare de nivel înalt sunt independente de mașină (microarhitectura hardware) pe care se procesează. Nivelul hardware este abstractizat din punct de vedere al limbajului (operațiile nu depind de arhitectura setului de instrucțiuni – ISA) [Patt03, Zah04].
- *C* este primul limbaj de nivel mediu (considerat de nivel înalt de către alţi cercetători) destinat creării de sisteme de operare (anterior acestuia sistemele de operare erau scrise în limbaje de asamblare). De asemenea, în C este permisă manipularea structurilor hardware ale calculatoarelor (regiştrii, memorie, porturi). C este un limbaj de programare standardizat, compilat, implementat pe marea majoritate a platformelor de calcul existente azi.
- Este apreciat pentru eficienţa codului obiect pe care îl poate genera, și pentru portabilitatea sa. A fost dezvoltat la începutul anilor 1970 de Brian Kernighan și Dennis Ritchie, care aveau nevoie de un limbaj simplu și portabil pentru scrierea nucleului sistemului de operare UNIX. Sintaxa limbajului C a stat la baza multor limbaje create ulterior și încă populare azi: C++, Java, JavaScript, C#. *C* este un limbaj *case sensitive*.
- Instrucţiunile din C care rezolvă o problemă sunt mult mai puţine decât cele ale limbajului asamblare aferent calculatorului LC-3 care rezolvă aceeaşi problemă [Patt03]. De asemenea, LC-3 nu asigură instrucțiuni de înmulţire și împărţire. Limbajele de nivel înalt sunt mai expresive decât cele de nivel mediu sau jos (asamblare). Sunt folosite simboluri pentru variabile și expresii simple pentru structuri de control (if-else, switch-case) sau repetitive (for, while, do-while). Permite compilarea condiţionată. Deşi limbajul C oferă o libertate foarte mare în scrierea codului, acceptând multe forme de scriere care în alte limbaje nu sunt permise, acest lucru poate fi și un dezavantaj, în special pentru programatorii fără experienţă.

Compilare vs. Interpretare

Programele de nivel înalt (*High Level Languages* – HLL) pot fi translatate în instrucțiuni maşină (ISA corespunzătoare arhitecturii hardware pe care se va procesa) prin două metode:

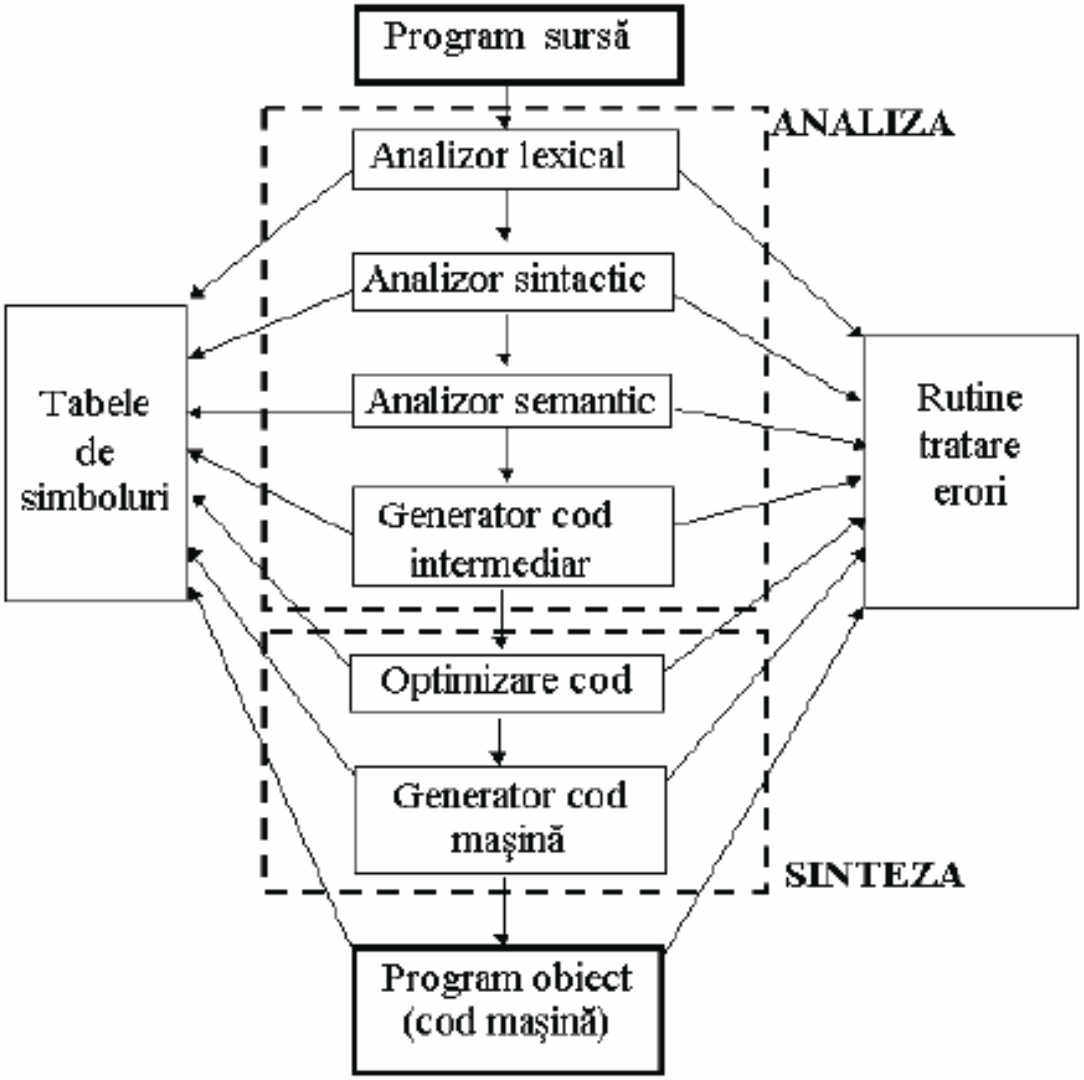
- **Interpretare:** Codul sursă HLL este „*interpretat*” de o maşină virtuală (cele mai cunoscute sunt maşinile virtuale Java – JVM) care translatează „*secţiuni*” din codul sursă în limbaj maşină și îl execută direct pe arhitectura gazdă, trecând apoi la următoarea *secţiune* de cod sursă. Dintre avantajele interpretării faţă de metodele tradiţionale de compilare s-ar menţiona simplitatea implementării hardware dar şi faptul că nu necesită o zonă mare de memorie pentru stocarea programului compilat. Principalul dezavantaj îl reprezintă viteza scăzută de execuție a aplicaţiei, dar şi necesitatea existenţei în momentul interpretării atât a codului sursă HLL cât şi a maşinii virtuale care realizează interpretarea.
- **Compilare:** Compilarea directă translatează codul sursă (poate fi Java, C, C++, Fortran, Pascal) în instrucțiuni maşină, direct executabile pe un procesor ţintă, dar nu le execută ca în cazul interpretoarelor. Orice modificare a codului sursă necesită recompilare. Procesul este realizat static şi poate îngloba tehnici de optimizare de tip analiza dependenţelor de date dintre instrucțiuni, analiză interprocedurală. Se caracterizează printr-o lipsă de portabilitate.

Descrierea componentelor unui compilator

Compilatorul translatează un program sursă scris într-un limbaj de nivel înalt (C, Pascal) în acelaşi program - obiect, de regulă scris în limbaj de asamblare.

În literatura de specialitate, este denumită *fază* a unui compilator o succesiune de operaţiuni prin care un program de la intrare suferă anumite modificări.

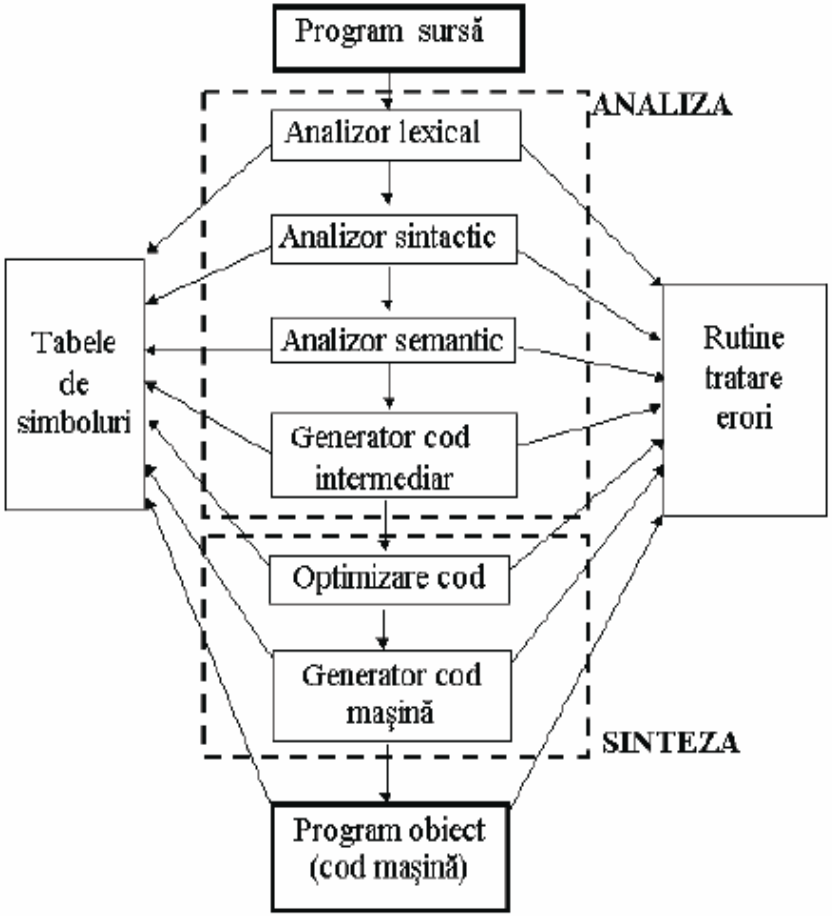
Prin *trecerea* executată de un compilator se înţelege o citire a programului dintr-un fişier, transformarea lui conform fazelor prestabilite și scrierea rezultatului în alt fişier(de ieșire).



Descrierea componentelor unui compilator

Figura ilustrează cele două faze majore ale procesului de compilare propriu-zisă:

- **analiza** (de tip "*front end*") - în care se identifică părţile constituente fundamentale ale programului (variabile, expresii, declaraţii, definiţii, apeluri de funcții) şi se construieşte o reprezentare internă a programului original, numită *„cod intermediar"* (*analiza lexicală* produce un -> şir de atomi lexicali -> *analiza sintactică* generează -> arborele sintactic -> *analiză semantică* construieşte o reprezentare a programului sursă în-> *cod intermediar*). Este dependentă de limbajul de nivel înalt şi nu de maşina pe care se va procesa.
- **sinteza** (de tip "*back end*") - generează cod maşină eventual optimizat. Se disting două etape
  - optimizare cod intermediar (scheduling) pentru o anumită maşină;
  - generare de cod maşină (generare cod într-o gamă variată de formate: *limbaj maşină absolut*, *limbaj maşină relocabil* sau *limbaj de asamblare* urmat de alocare de resurse). Această fază este puternic dependentă de maşină pe care se va executa codul obiect generat.



Descrierea componentelor unui compilator

Înaintea primei faze din cadrul procesului de compilare trebuie realizată de cele mai multe ori o **preprocesare**. Preprocesorul translatează un program al cărui limbaj sursă este de nivel înalt (C, Pascal) într-un program destinaţie (obiect) scris tot într-un limbaj de nivel înalt C. Practic are loc interpretarea *directivelor de preprocesare* (încep cu # - *include, define*, *if !defined*(**…**)**…***endif*). De exemplu, preprocesorul trebuie să introducă conţinutul fişierului <*stdio.h*> în codul sursă al aplicaţiei create acolo unde apare respectiva directivă de preprocesare, sau, să înlocuiască anumite valori constante declarate ca şiruri de caractere cu valorile numerice aferente, dacă acest lucru se specifică printr-o directivă de tip *#define* în cadrul programului.

- **Analiza lexicală** reprezintă prima fază a procesului de compilare, şi care este responsabilă de transformarea programului sursă văzut ca o succesiune de caractere (text) într-o succesiune de atomi lexicali și de atribute ale lor. Conform definiţiei din , un atom lexical este o entitate indivizibilă a unui program - identificatori de variabile/funcții, constante de diverse tipuri, operatori, cuvinte cheie (if, else, switch/case…), delimitatori. Atributele atomilor lexicali sunt: clasă, tip, lungime, loc în tabela de simboluri, dacă a fost definit sau nu, etc. Clasa atributului se referă la faptul că este cuvânt-cheie, identificator de variabilă, constantă de tip întreg sau real etc.

Descrierea componentelor unui compilator

Toate fazele unui compilator dispun de **rutine de tratare a erorilor** şi lucrează în comun cu una sau mai multe tabele de simboluri în care se păstrează informații despre cuvintele cheie ale limbajului, identificatorii de variabile (tip, lungime, adresă stocare în memorie etc.), etichete de instrucțiuni, identificatori de proceduri și funcții și alte informații utile despre programul în curs de compilare.

Tabela de simboluri este creată în faza de analiză a programului sursă şi folosită în scopul validării numelor simbolice facilitând generarea de cod . Tabela de simboluri realizează o asociere simbolică între numele (identificatorul) unei variabile și caracteristicile acesteia (tip, domeniu de vizibilitate, deplasament faţă de începutul zonei de date statice sau dinamice, stivă).

Compilatorul foloseşte tabela de simboluri pentru a urmări domeniul de utilizare și valabilitate a unui nume şi de a adăuga informaţii despre un nume.

Tabela de simboluri este cercetată de fiecare dată când un nume este întâlnit în textul sursă. Mecanismul asociat tabelei de simboluri trebuie să permită adăugarea a noi intrări și găsirea informaţiilor existente în mod eficient.

Tabela de simboluri este similară celei generate de asamblor însă cuprinde mai multe informații decât aceasta. În asamblare, toţi identificatorii (variabilele) erau etichete (*labels*) şi informaţiile reprezentate de acestea erau adrese. Tabela de simboluri conține informații legate de toate variabilele din program

Descrierea componentelor unui compilator

Generatorul de cod constituie faza finală a unui compilator. Primeşte la intrare reprezentarea intermediară a programului sursă împreună cu informaţia din tabela de simboluri - folosită la determinarea run-time a adresei obiectelor de date, desemnate de nume în reprezentarea intermediară, și produce la ieşire un program obiect echivalent.

Codul de ieşire trebuie să fie corect și de înaltă calitate, aceasta însemnând că el trebuie să folosească eficient resursele maşinii pentru a putea fi procesat cât mai rapid.

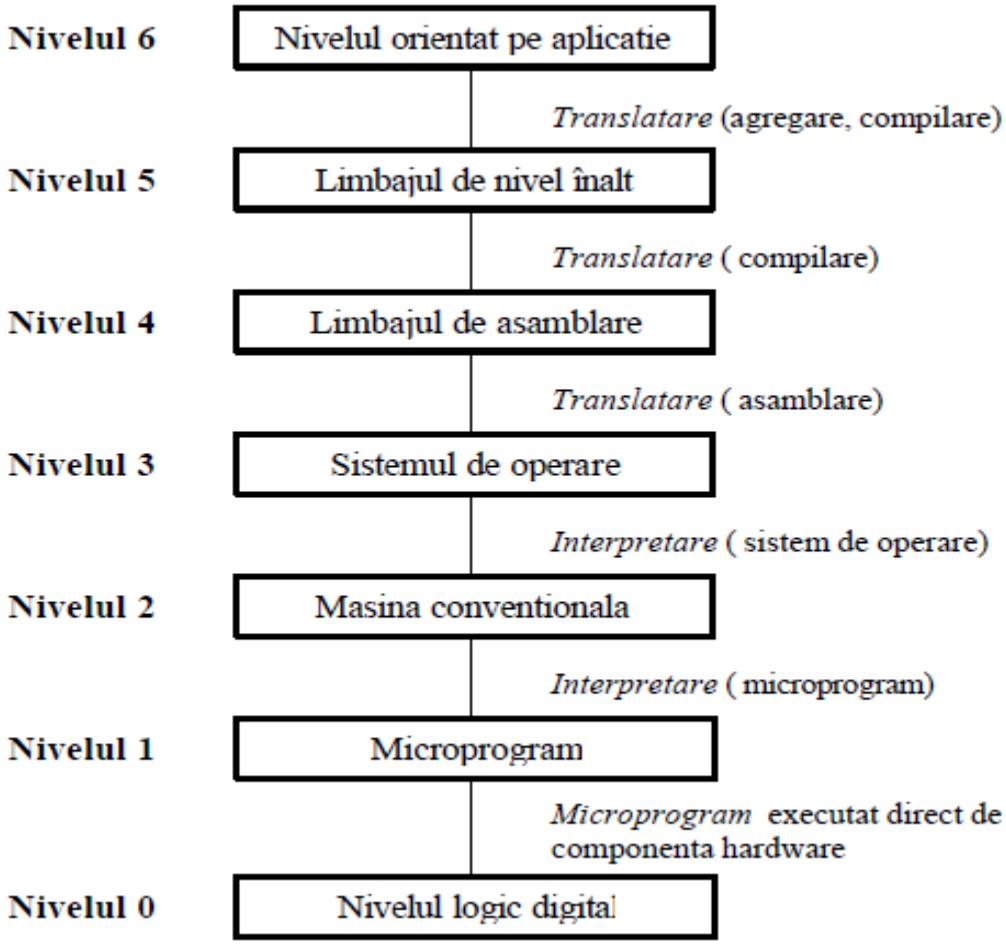
Ieşirile generatorului de cod se găsesc într-o gamă variată de forme:

- *limbaj de asamblare*,
- *limbaj maşină relocabil* sau
- *limbaj maşină absolut*.

Producând un program într-un limbaj de asamblare, caracterizat de o mai mare lizibilitate, generarea codului se realizează mult mai uşor. Pot fi generate instrucțiuni simbolice precum şi utilizate facilităţile de macro-asamblare ajutătoare la generarea de cod. Producând un program în limbaj maşină absolut, există avantajul că programul poate fi plasat într-o locaţie fixă a memoriei și poate fi imediat executat. Producând un program în limbaj maşină relocabil, acesta permite subprogramelor să fie compilate separat (similar cu fişierul *Makefile* din cadrul setului de instrumente SimpleScalar 3.0). O mulţime de module-obiect relocate pot fi *legate* (*linked*) împreună și apoi încărcate pentru execuție de către un *loader*. Câştigul de flexibilitate care dă posibilitatea compilării separate a subrutinelor şi apelarea altor programe compilate anterior dintr-un modul obiect este tributar costurilor suplimentare pentru legare şi încărcare.

Programarea în limbaj de asamblare

- Limbajul de asamblare permite accesul la unele dintre resursele fizice ale masinii în condițiile utilizarii unor mnemonice mai usor de manevrat decât codurile numerice asociate instrucțiunilor limbajului mașină și are nivelul 4 – nivelul limbajului de asamblare:



- Există câte un astfel de limbaj specific fiecărei familii de procesoare.

Programarea în limbaj de asamblare

Structura unui program scris în assembler

Exemplu TASM:

```
.model MMMMM
.stack NNNNN
.nume
      ; Declaratii de date
.code
      ; Definitii de proceduri
start:
      ; urmeaza:
      ;  initializare ds, es
      ;  Program principal
      ;  exit dos
end start
```

Programarea în limbaj de asamblare
Definirea segmentelor

Declaraţia:

```
nume_seg SEGMENT [tip_aliniere][tip_combinare][clasa_seg]
 …
nume_seg ENDS
```

Un registru de segment (DS, ES sau SS) este iniţializat cu un segment declarat. Aceasta se face de programator explicit în cadrul programului, utilizând numele segmentului:

```
mov  ax, nume_seg
mov  ds, ax
```

- *tip_aliniere* - poate fi BYTE, WORD, PARA, PAGE (implicit este PARA), şi arată că adresa de început a zonei de memorie rezervatăsegmentului este divizibilă cu 1/2/16/256.

- *tip_combinare* - este un parametru folosit de editorul de legături ce indică raportul dintre acest segment şi segmentele definite în alte module obiect.

Programarea în limbaj de asamblare

Definirea segmentelor

- *tip_combinare* poate fi:

- PUBLIC – arată că acest segment va fi concatenat cu alte segmente cu acelaşi nume, din alte module obiect, rezultând un singur modul cu acest nume;
- COMMON - precizeazăcăacest segment şi alte segmente cu acelaşi nume din alte module vor fi suprapuse, deci vor începe de la aceeaşi adresă. Lungimea unui astfel de segment va fi lungimea celui mai mare segment dintre cele suprapuse;
- AT <expresie> - segmentul va fi încărcat în memorie la adresa fizică absolutăde memorie reprezentată de valoarea expresiei, care este o valoare de 16 biţi;
- MEMORY - segmentul curent de acest tip va fi aşezat în memorie în spaţiul rămas disponibil după aşezarea celorlalte segmente;
- STACK - va concatena toate segmentele cu acelaşi nume, pentru a forma un segment unic; referirea acestui segment se va face prin SS:SP. Registrul SP va fi iniţializat automat cu lungimea stivei.

- *clasa_seg* - este un parametru declarat între apostrofuri și permite stabilirea modului în care se aşează în memorie segmentele unui program. Două segmente cu aceeaşi clasă vor fi aşezate în memorie la adrese succesive. Asocierea segmentelor cu regiştrii de segment se realizează cu pseudo-instrucţiunea ASSUME.

ASSUME reg_seg : nume_seg

Programarea în limbaj de asamblare

Definirea segmentelor

Unde `reg_seg` poate fi unul dintre regiştrii CS, DS, ES sau SS, iar `nume_seg` este numele unui segment sau al unui grup de segmente, care va fi adresat de registrul respectiv.

Definirea grupurilor de segmente se realizează cu directiva GROUP, care are următoarea formă:

```
nume_grup GROUP lista_nume_segmente
```

unde nume_grupeste numele grupului de segmente, ce va fi folosit pentru a determina adresa de segment utilizată pentru referirea în cadrul grupului de segmente, iar lista_nume_segmentepoate conţine nume de segmente sau expresii de forma:

```
SEG nume_variabila
SEG nume_eticheta
```

Într-o astfel de listă nu poate să apară numele unui alt grup.

Exemplu:

Programarea în limbaj de asamblare

Definirea segmentelor

```
     nume1 segment
     vt1 db 5
     nume1 ends
    nume2 segment
    vt2 dw 25
    nume2 ends
     nume3 segment
     vt3 dw 100
     nume3 ends
    dgrup group nume1, nume2
    cod segment
     assume cs:cod, ds:dgrup, es:nume3
start:   mov ax, dgrup    mov ds, ax
     mov ax, nume3
     mov es, ax
     ;referiri la date
   mov bx, vt1 ;se fol DS pentru ca a fost asociat cu dgrup
   add v3, bx  ;se fol ES ptr ca a fost asociat cu segmentul nume3
    cod ends
    end start
```

Programarea în limbaj de asamblare

Forma simplificată de definire a segmentelor

Tipul simplificat pentru definirea segmentelor respectă acelaşi format ca şi programele dezvoltate în limbaje de nivel înalt.

`.MODEL` *`tip_model`*

Această directivă specifică dimensiunea şi modul de dispunere a segmentelor în memorie. *tip_model* poate fi:

- `tiny` - toate segmentele (date, cod, stivă) formeazăun singur segment de cel mult 64KB. Apelurile de procedură şi salturile sunt de tip NEAR şi se folosesc adrese efective (offset) pentru accesarea datelor;

- `small` - datele şi stiva formeazăun segment şi codul un alt segment. Fiecare din acestea va avea dimensiunea maxima de 64KB. Apelurile de procedură şi salturile sunt de tip NEAR şi se folosesc adrese efective (offset) pentru accesarea datelor;

- `medium` - datele şi stiva sunt grupate într-un singur segment (cel mult egal cu 64KB), dar codul poate fi în mai multe segmente, deci poate depăşi 64KB. Apelurile de procedură şi salturile sunt implicit de tip FAR şi se folosesc adrese efective (offset) pentru accesarea datelor;

Programarea în limbaj de asamblare
Forma simplificată de definire a segmentelor

(continuare)

`.MODEL` *`tip_model`*

- `compact` - codul generat ocupăcel mult 64KB, dar datele şi stiva pot fi în mai multe segmente (pot depăşi 64KB). Apelurile de procedură şi salturile sunt de tip NEAR şi se folosesc adrese complete (segment şi offset) pentru accesarea datelor aflate în alte segmente;

- `large` - atât datele cât şi codul generat pot depăşi 64KB. Apelurile de procedură şi salturile sunt implicit de tip FAR şi se folosesc adrese complete (segment şi offset) pentru accesarea datelor aflate în alte segmente;

- `huge` - este asemănător modelului large, dar structurile de date pot depăşi 64KB. La modelele compact şi large, o structurăcompactăde date (de tip tablou) nu poate depăşi limitele unui segment fizic (64KB); la modelul huge, aceastărestricţie dispare.

Programarea în limbaj de asamblare

Forma simplificată de definire a segmentelor

`.STACK [dimensiune]`

Alocă o zonă de memorie de marimea specificată prin [dimensiune] pentru segmentul de stivă. Dacă nu se specifică parametrul, aceasta va fi implicit de 1KB.

`.CODE [nume]`

Directiva aceasta precede segmentul de cod. Încărcarea adresei acestui segment în registrul CS se face automat de către sistemul de operare, la încărcarea segmentului pentru execuţie. Opţional se pot asocia nume (maxim 6 caractere) pentru segmentele de cod.

`.DATA`

Directiva precede segmentul de date. Utilizatorul trebuie să iniţializeze în mod explicit registrul de segment DS cu adresa segmentului de date. Simbolul `@nume` primeşte adresa segmentului de date după link-editare.

Programarea în limbaj de asamblare

Structuri

Colecţiile de date plasate congruent în memorie reprezintă structurile de date. Acestea sunt identificate printr-un nume unic, sub care se grupează datele respective.
Structura are o dimensiune (size) determinată de suma dimensiunilor câmpurilor componente. Forma generală este:

```
nume_structura STRUC
nume_membru definitie_date
nume_structura ENDS
```

De observat că structura (de fapt tipul structurii) se defineşte fără a se rezerva spaţiu de memorie. Numele membrilor structurii trebuie să fie diferite, chiar dacă aparţin unor structuri distincte.
Exemplu de definiţie a unei structuri:

```
alfa struc
 a db  ?
 b dw  1111H
 c dd  1234
alfa ends
```

O nouă structură definită este interpretată ca un tip nou de date, care poate apoi participa la definiţii de date, cu rezervare de spaţiu de memorie.

Programarea în limbaj de asamblare

Structuri

De exemplu, putem defini o structurăde tipul alfa, cu numele x:

```
x alfa <, , 777>
```

unde alfa este tipul de date, x este numele variabilei, iar între paranteze unghiulare se trec valorile de iniţializare pentru câmpurile structurii x. Prezenţa virgulelor din lista de valori iniţiale precizează că primii doi membrii sunt iniţializaţi cu valorile implicite de la definiţia tipului alfa, iar al treilea membru este iniţializat cu valoarea 777. Astfel, definiţia variabilei x este echivalentă cu secvenţa de definiţii:

```
a db  ?
b dw  1111H
c dd  777
```

Avantajul structurilor este dat de accesul la membr, asemănător limbajelor de nivel înalt.
Pentru a încărca în SI membrul b al structurii x:

```
mov si,  x.b
```

Pentru a defini un tablou de 5 structuri de tip alfa:

```
tab alfa  5 dup (<, , 10>)
```

unde primii 2 membri sunt iniţializaţi cu valorile de definiţie implicite iar al treilea cu valoarea 10.

Programarea în limbaj de asamblare
Înregistrări

Înregistrările sunt structurile împachetate din limbajele de nivel înalt. Acestea sunt definiţii de câmpuri de biţi de lungime maximă 8 sau 16.
Forma generală este:

```
nume_inregistrare RECORD nume_camp:valoare [= expresie], …
```

unde
- `valoare` : numărul de biţi pe care se memorează câmpul respectiv. Poate fi folosită o expresie a cărei valoare să fie asociată câmpului respectiv, pe numărul de biţi precizat. La fel ca la structuri, numele câmpurilor trebuie să fie distincte, chiar dacă aparţin unor înregistrări diferite.

Exemplu:

```
gamma record m:7, n:4, p:5
```

defineşte un şablon de 16 biţi, m pe primii 7 biţi (mai semnificativi), n pe următorii 4 biţi, şi p pe ultimii 5 biţi (mai puţin semnificativi). La fel ca la structuri, putem acum defini variabile de tipul înregistrării gamma:

```
var gamma <5, 2, 7>
```

unde valorile dintre parantezele unghiulare iniţializează cele trei câmpuri de biţi. Această definiţie este echivalentă cu definiţia explicită:

```
var dw 0000101001000111B
```

Programarea în limbaj de asamblare
Înregistrări

`MASK`

Operatorul MASK primeşte un nume de câmp, returnând o mască cu biţii 1 pe poziţia câmpului respectiv şi 0 în rest.
Exemplu :

```
MASK n
```

Expresia este echivalent ăcu constanta binară 0000000111100000B.

`WIDTH`

Acest operatorul are ca parametru un nume de înregistrare sau un nume de câmp şi returnează numărul de biţi ai înregistrării sau ai câmpului respectiv. De exemplu, secvenţa:

```
mov al, width gamma
mov bl, width n
```

va încărca în AL 16 şi în BL valoarea 4.

Programarea în limbaj de asamblare

Macroinstrucțiuni

Unele definiții simbolice pot identifica secvențe de program, cum ar fi instrucţiuni, definiţii de date, directive, etc., prin asocirerea cu un nume. Acestea sunt macroinstrucţiunile care sunt substituite înaintea asamblării, cu segmentul respectiv, expandate la fiecare utilizare.

Forma generală:

```
nume_macro MACRO [p1, p2, …,pn]
…
ENDM
```

Unde
[p1, p2, …,pn] sunt parametri opţionali formali.

Apelul unei macroinstrucţiuni se face prin simpla scrierea în program a numelui macroinstrucţiunii. Dacă macroinstrucţiunuea are parametrii, apelul se face prin specificarea numelui urmat eventual de o listă de parametri actuali:

```
nume_macro a1, a2, …, an
```

La substituție, pe lângă expandarea propriu-zisă, se va înlocui fiecare parametru formal cu parametrul actual corespunzător.

Programarea în limbaj de asamblare

Întreruperi

Întreruperea presupune ieşirea din secvența programului în curs de execuţie şi transferul controlului la o rutină specifică de tratare. Mecanismul folosit este de tip apel de procedură, ceea ce înseamnă revenirea în programul întrerupt, după terminarea rutinei de tratare. Pe lângă rutinele de tratare ale sistemului de operare, pot fi folosite şi propriile rutine.

Întreruperile software apar ca urmare a execuţiei unor instrucţiuni, cum ar fi INT, INTO, DIV, IDIV. Întreruperile hardware externe sunt provocate de semnale electrice care se aplică pe intrările de întreruperi ale procesorului, iar cele interne apar ca urmare a unor condiţii speciale de funcţionare a procesorului (cum ar fi execuţia pas cu pas a programelor).

Într-un sistem cu procesor 8086, pot exista maxim 256 de întreruperi. Fiecare nivel de întrerupere poate avea asociată o rutină de tratare (procedură de tip far). Adresele complete (4 octeţi) ale acestor rutine sunt memorate în tabela vectorilor de intrerupere, pornind de la adresa fizică 0 până la 1KB. Intrarea în tabelă se obţine înmulţind codul întreruperii cu 4. Rutina de tratare a întreruperii se termină obligatoriu cu instrucţiunea IRET.

Programarea în limbaj de asamblare

Întreruperi

Observaţii :

- citirea unui caracter de la tastatură se realizează folosind funcţia 01H a întreruperii 21H. Se încarcă în registrul AH valoarea 01H, se apelează întreruperea 21H, care va pune codul ASCII al caracterului citit în AL;
- afişarea unui caracter pe ecran se realizează folosind funcţia 02H a întreruperii 21H. Se încarcă în registrul AH valoarea 02H. Se încarcă în DL codul ASCII al caracterului care trebuie afişat şi apoi se apelează întreruperea 21H, care va afişa caracterul pe ecran;
- afişarea unui şir de caractere pe ecran (şirul se termină cu caracterul $, care are codul ASCII 24H) se realizează folosind funcţia 09H a întreruperii 21H. Se încarcă în registrul AH valoarea 09H. Se încarcă în DS adresa de segment a şirului care trebuie afişat, şi în DX adresa efectivă (offsetul). Se apelează întreruperea 21H, care va afişa şirul pe ecran.
- pentru trecerea la linie nouă se afişează şirul de caractere:

nl db 13, 10, 24H

unde 13, 10 reprezintă codul ENTER-ului.

Programarea în limbaj de asamblare – Familia de procesoare Intel x86

- Cuprinde seria de procesoare 8086, 8088, 80286, '386, '486, Pentium, Pentium Pro, şi variantele ultime Pentium II,III şi IV. Toate aceste procesoare împărtăşesc o arhitectură comună și utilizează acelaşi set de instrucțiuni de asamblare. Îmbunătăţirile arhitecturale aduse în decursul timpului la noile versiuni de procesoare s-au făcut în aşa fel încât să se menţină compatibilitatea cu imaginea iniţială a unui procesor I8086.
- Chiar dacă în interior un procesor din ultima generaţie diferă semnificativ de structura primului procesor din familie, acest lucru este vizibil în mică măsură pentru un programator. De aceea nu se mai vorbeşte de o anumită arhitectură particulară de procesor ci de o arhitectură corespunzătoare unui set de instrucțiuni de asamblare. Astfel pentru seria de procesoare Intel pe 16 și 32 de biţi se operează cu imaginea arhitecturală denumită ISA x86 (Instruction Set Architecture).
- În acelaşi mod pentru procesoarele pe 64 de biţi s-a introdus arhitectura ISA 64. Această arhitectura diferă semnificativ de cea a procesoarelor pe 16 şi 32 de biţi, iar setul instrucțiunilor de asamblare este diferit. Astfel un program scris pentru arhitectura ISA 64. nu mai este compatibil la nivel de coduri de instrucțiune cu arhitectura ISA x86. Este interesant de observat că Intel a propus această nouă arhitectură chiar înainte de lansarea pe piaţă a unui procesor pe 64 de biţi.
- În ceea ce priveşte arhitectura ISA x86, aceasta a suferit anumite modificări impuse de introducerea, în versiunile mai noi de procesoare, a unor elemente arhitecturale şi funcţionale noi. Însă aceste elemente noi nu modifică componentele existente deja, iar programele scrise pentru primele versiuni de procesoare rulează și pe versiunile noi fără nici o modificare.
- Cunoaşterea arhitecturii ISA x86 este strict necesară pentru a putea scrie programe în limbaj de asamblare.

Programarea în limbaj de asamblare – Familia de procesoare Intel x86

Componentele

În continuare se prezintă acele componente ale unui procesor care sunt vizibile și accesibile pentru un programator în limbaj de asamblare. Astfel, un procesor dispune de un set de registre interne folosite pentru păstrarea temporară a datelor, a adreselor sau a instrucţiunilor. Există registre generale folosite în majoritatea operațiilor aritmetico-logice și registre speciale, care au o destinaţie specială.

- La arhitectura ISA x86 pe 16 biţi există 8 registre generale pe 16 biţi, registre denumite în felul următor: AX, BX, CX, DX, SI, DI, BP, SP. Dintre acestea primele patru se folosesc cu precădere în operații aritmetico-logice pentru păstrarea operanzilor, iar următoarele patru mai ales pentru păstrarea și calculul adreselor de operanzi. Primele patru registre se pot adresa şi pe jumătate, adică la nivel de octet. Astfel fiecare registru are o parte superioară "HIGH" (biţii D15-D8) și o parte inferioară "LOW" (biţii D7-D0). Denumirea lor este AH şi AL pentru registrul AX, BH şi BL pentru registrul BX şi aşa mai departe.
- La procesoarele pe 32 de biţi (începând de la '386) aceste registre s-au extins la 32 de biţi. Astfel registrele pot fi adresate pe 32 de biţi (cu denumirile EAX,EBX, ECX, ... ESP) sau în mod uzual pe 16 sau 8 biţi. Partea superioara a unui registru pe 32 de biţi (D31-D16) nu se poate adresa individual.
- În principiu registrele generale respectă principiul de ortogonalitate, adică pentru majoritatea operațiilor oricare registru este utilizabil. La unele procesoare (ex: procesoarele Motorola) acest principiu este respectat strict. În cazul procesoarelor Intel anumite operații speciale impun utilizarea anumitor registre, ceea ce înseamnă că există o oarecare specializare între registre. Această specializare decurge de multe ori și din denumirea registrelor.

Programarea în limbaj de asamblare – Familia de procesoare Intel x86

Componentele (cont.)

Astfel:

- registrul **AX** (**EAX** în varianta pe 32 de biţi) – se foloseşte ca registru "acumulator" în majoritatea operațiilor aritmetice și logice, adică păstrează unul dintre operanzi și apoi rezultatul operaţiei; la operațiile de înmulţire și împărţire în mod obligatoriu primul operand şi rezultatul se păstrează în acest registru
- registrul **BX** (**EBX**) – se foloseşte pentru în operații aritmetico-logice sau pentru calculul adresei operandului la "adresarea bazată"
- registrul **CX** (**ECX**) – se foloseşte pentru operații aritmetico-logice sau în mod implicit, la anumite instrucțiuni (ex: instrucţiuni de buclare), pe post de contor
- registrul **DX** (**EDX**) – se foloseşte pentru operații aritmetico-logice sau pentru păstrarea adresei porturilor la instrucțiunile de intrare/ieşire; de asemenea la operaţiile de înmulţire și împărţire se foloseşte ca extensie a registrului acumulator
- registrele **SI** și **DI** (**ESI**, **EDI**) – denumite și registre index, se folosesc cu precădere pentru calculul adresei operanzilor la "adresarea indexată"; SI adresează elementele şirului sursă (source index), iar DI elementele şirului destinaţie (destination index)
- registrul **BP** (**EBP**) – se foloseşte cu precădere pentru calculul adresei operanzilor la "adresarea bazată", alături de registrul *BX*
- registrul **SP** (**ESP**) – se foloseşte aproape în exclusivitate pentru adresarea stivei (stack pointer); conţinutul registrului se incrementează și se decrementează automat la orice operaţie cu stiva

Programarea în limbaj de asamblare – Familia de procesoare Intel x86

Componentele (cont.)

- În afara registrelor generale un procesor mai are și registre speciale sau de control. La procesoarele Intel numai o parte din aceste registre sunt vizibile și accesibile pentru programator. Aceste registre controlează regimul de lucru al procesorului, sau permit efectuarea unor operații speciale de manipulare a spaţiului de memorie.
- Arhitectura ISA x86 operează cu următoarele registre speciale:
- registrul **PC** – "program counter" – păstrează adresa instrucţiunii care urmează; nu este adresabil direct (prin nume) dar conținutul său se poate modifica prin execuția instrucțiunilor de salt
- registrul **PSW** – "program status word" – păstrează indicatorii de stare ai procesorului;
  - o parte din indicatori caracterizează rezultatul obţinut în urma unei anumite instrucţiuni:
    - *ZF* – rezultat 0,
    - *SF* – semnul rezultatului
    - *OF* – "overflow" indică o depăşire de capacitate la ultima operaţie aritmetică
    - *PF* – indicator de paritate (arată paritatea rezultatului)
    - *CF* – indicator de transport ("carry"), arată dacă s-a generat un transport
    - *AC* – indicator de transport auxiliar, arată dacă după primii 4 biţi s-a generat un transport
  - o parte din indicatori controlează modul de lucru al procesorului:
    - *IF* – indicator de întrerupere (interrupt"), dacă este setat (1 logic) atunci se blochează toate întreruperile mascabile, în caz contrar (0 logic) se validează
    - *DF* – indicatorul de direcţie, arată direcţia de parcurgere a şirurilor de caractere la instrucțiunile pe şiruri (în ordine ascendentă sau descendentă a adreselor)

Programarea în limbaj de asamblare – Familia de procesoare Intel x86

Componentele (cont.)

- registrele de segment – se utilizează pentru calculul adresei operanzilor şi a instrucţiunilor; cu ajutorul lor memoria este divizată în segmente; există 4 registre segment în versiunea iniţială ISA x86 şi 6 registre segment în versiunea pe 32 de biţi:
  - registrul **CS** – registrul segment de cod, folosit pentru adresarea instrucțiunilor (a codurilor); acest registru păstrează adresa de început (descriptorul în varianta extinsă) pentru segmentul de cod (segmentul unde se află programul)
  - registrul **DS** – registrul segment de date, folosit pentru adresarea operanzilor din memorie; acest registru păstrează adresa de început (sau descriptorul) pentru segmentul de date
  - registrul **SS** – registrul segment de stivă, folosit pentru adresarea memoriei stivă; păstrează adresa de început (descriptorul) segmentului unde se află organizata stiva
  - registrul **ES** – registrul extra-segmentului de date, folosit pentru păstrarea adresei celui de al doilea segment de date
  - registrele **FS** și **GS** – registre segment introduse începând de la versiunea '386 şi care se folosesc pentru adresarea operanzilor

Programarea în limbaj de asamblare – Familia de procesoare Intel x86

Componentele (cont.)

- Versiunile mai noi de procesoare conțin şi alte registre speciale, dar acestea au o importanţă mai mică pentru un programator obişnuit. Ele vor fi introduse pe parcurs în momentul în care acest lucru va fi necesar.
- Un limbaj de asamblare conține instrucțiuni corespunzătoare unor operații simple care sunt direct interpretate şi executate de procesor. Fiecărei instrucţiuni din limbajul de asamblare îi corespunde în mod strict un singur cod executabil. În contrast, unei instrucțiuni dintr-un limbaj de nivel înalt (ex: C, Pascal, etc.) îi corespunde o secvenţă de coduri (instrucțiuni în cod maşină). Un anumit limbaj de asamblare este specific pentru un anumit procesor sau eventual pentru o familie de procesoare. Instrucţiunile sunt în directă corelaţie cu structura internă a procesorului. Un programator în limbaj de asamblare trebuie să cunoască această structură precum şi tipurile de operații permise de structura respectivă.
- Un program în asamblare scris pentru o anumită arhitectură de procesor nu este compatibil cu un alt tip de procesor. Pentru implementarea unei aplicaţii pe un alt procesor programul trebuie rescris. În schimb programele scrise în limbaj de asamblare sunt în general mai eficiente atât în ceea ce priveşte timpul de execuţie cât şi spaţiul de memorie ocupat de program. De asemenea, programarea în limbaj de asamblare dă o mai mare flexibilitate şi libertate în utilizarea resurselor unui calculator. Cu toate acestea astăzi utilizarea limbajului de asamblare este mai puţin frecventă deoarece eficienţa procesului de programare este mai scăzută, există puţine structuri de program şi de date care să uşureze munca programatorului, iar programatorul trebuie să cunoască structura procesorului pentru care scrie aplicaţia. În plus programele nu sunt portabile, adică nu rulează şi pe alte procesoare.

Directive și instrucțiuni ASM

- Un program scris în limbaj de asamblare conține instrucțiuni și directive.
- Instrucţiunile sunt traduse în coduri executate de procesor; ele se regăsesc în programul executabil generat în urma compilării și a editării de legături.
- Directivele sunt construcţii de limbaj ajutătoare care se utilizează în diferite scopuri (ex: declararea variabilelor, demarcarea segmentelor și a procedurilor, etc.) și au rol în special în fazele de compilare și editare de legături.
- O directivă nu se traduce printr-un cod executabil şi în consecinţă NU se execută de către procesor.

Directive și instrucțiuni ASM
Sintaxa unei instrucțiuni în limbaj de asamblare

O instrucţiune ocupă o linie de program și se compune din mai multe câmpuri, după cum urmează (parantezele drepte indică faptul că un anumit câmp poate să lipsească):

[<eticheta>:] [<mnemonica> [<parametru_1> [,<parametru_2>]] [;<comentariu>]

- <eticheta> - este un nume simbolic (identificator) dat unei locații de memorie care conține instrucţiunea care urmează; scopul unei etichete este de a indica locul în care trebuie să se facă un salt în urma executării unei instrucțiuni de salt; eticheta poate fi o combinație de litere, cifre şi anumite semne speciale (ex: _), cu restricţia ca prima cifră să fie o literă
- <mnemonica> - este o combinație de litere care simbolizează o anumită instrucțiune (ex: add pentru adunare, mov pentru transfer, etc.); denumirile de instrucțiuni sunt cuvinte rezervate şi nu pot fi utilizate în alte scopuri
- <parametru_1> - este primul operand al unei instrucțiuni și în acelaşi timp și destinaţia rezultatului; primul parametru poate fi un registru, o adresă, sau o expresie care generează o adresă de operand; adresa operandului se poate exprima și printr-un nume simbolic (numele dat unei variabile)
- <parametru_2> - este al doilea operand al unei instrucțiuni; acesta poate fi oricare din variantele prezentate la primul operand şi în plus poate fi și o constantă

Directive și instrucțiuni ASM
Sintaxa unei instrucțiuni în limbaj de asamblare (cont.)

- <comentariu> - este un text explicativ care arată intenţiile programatorului și efectul scontat în urma execuţiei instrucțiunii; având în vedere că programele scrise în limbaj de asamblare sunt mai greu de interpretat se impune aproape în mod obligatoriu utilizarea de comentarii; textul comentariului este ignorat de compilator; comentariul se considera până la sfârşitul liniei curente

Într-o linie de program nu toate câmpurile sunt obligatorii: poate să lipsească eticheta, parametrii, comentariul sau chiar instrucțiunea. Unele instrucțiuni nu necesită nici un parametru, altele au nevoie de unul sau doi parametri. În principiu primul parametru este destinaţia, iar al doilea este sursa.

Constantele numerice care apar în program se pot exprima în zecimal (modul implicit), în hexazecimal (constante terminate cu litera 'h') sau în binar (constante terminate cu litera 'b'). Constantele alfanumerice (coduri ASCII) se exprimă prin litere între apostrof sau text între ghilimele.

Clase de instrucțiuni

În această prezentare nu se va face o prezentare exhaustiva a tuturor instrucţiunilor cu toate detaliile lor de execuție. Se vor prezenta acele instrucțiuni care se utilizează mai des și au importanţă din punct de vedere al structurii și al posibilităţilor procesorului. Pentru alte detalii se pot consulta documentaţii complecte referitoare la setul de instrucțiuni ISA x86 (ex: cursul AoA – The Art of Assembly Language, accesibil pe Internet).

Directive și instrucțiuni ASM
Instrucţiuni de transfer

Instrucţiunile de transfer realizează transferul de date între registre, între un registru şi o locaţie de memorie sau o constantă se încarcă într-un registru sau locaţie de memorie. Transferurile de tip memorie-memorie nu sunt permise (cu excepţia instrucțiunilor pe şiruri). La fel nu sunt permise transferurile directe între două registre segment. Ambii parametrii ai unui transfer trebuie să aibă aceeaşi lungime (număr de biţi).

Instrucţiunea MOV

Este cea mai utilizată instrucțiune de transfer. Sintaxa ei este:

[<eticheta>:] MOV <parametru_1>, <parametru_2> [;<comentariu>]

unde:

- <parametru_1> = <registru> | <reg_segment>| <adresa_offset>| <nume_variabilă>| <expresie>
- <parametru_2> = <parametru_1>|<constantă>
- <registru> = EAX|EBX|.....ESP|AX|BX|....SP|AH|AL|....DL
- <expresie> = **[**[<registru_index>][+<registru_bază>][+<deplasament>]**]**

  ; aici parantezele drepte marcate cu bold sunt necesare
- <registru_index> = SI| DI |ESI | EDI
- <registru_bază> = BX|BP |EBX| EBP
- <deplasament> = <constantă>

Directive și instrucțiuni ASM
Instrucţiuni de transfer (cont.)

Exemple:

```
mov ax,bx                    et1:   mov ah, [și+100h]
mov cl, 12h                         mov al, 'A'
mov dx, var16                       mov și, 1234h
mov var32,eax                sf:    mov [și+bx+30h], dx
mov ds, ax                          mov bx, cs
```

Exemple de erori de sintaxă:

```
mov ax, cl           ; operanzi inegali ca lungime
mov var1, var2       ; ambii operanzi sunt locații de memorie
mov al, 1234h        ;  dimensiunea  constantei  este  mai  mare
decât cea a registrului
mov ds, es           ; două registre segment
```

Directive și instrucțiuni ASM
Instrucţiuni de transfer (cont.)

Instrucţiunea LEA, LDS şi LES

Aceste instrucțiuni permit încărcarea într-un registru a unei adrese de variabile. Prima instrucțiune LEA ("load effective address") încarcă în registrul exprimat ca prim parametru adresa de offset a variabilei din parametrul 2.

Următoarele două instrucțiuni încarcă atât adresa de offset cât și adresa de segment;

LDS încarcă segmentul în registrul DS, iar LES în ES.

```
LEA parametru_1>,<parametru_2>
LDS <parametru_1>, <parametru_2
```

- Exemple:

```
lea  si, var1        ; SI<= offset(var1)
lds bx, text         ; DS<= segment(text), BX<=offset(text)
lea di, [bx+100]     ; DI<= BX+100
```

Directive și instrucțiuni ASM
Instrucţiuni de transfer (cont.)

Instrucţiunea XCHG

Această instrucțiune schimbă între ele conținutul celor doi operanzi.

```
XCHG <parametru_1>, <parametru_2>
```

Atenţie: parametrul 2 nu poate fi o constantă.

Exemple:

```
xchg al,bh
xchg ax,bx
```

Directive și instrucțiuni ASM
Instrucţiuni de transfer (cont.)

Instrucţiunile PUSH şi POP

Cele două instrucțiuni operează în mod implicit cu vârful stivei.

Instrucţiunea PUSH pune un operand pe stivă, iar POP extrage o valoare de pe stivă și o depune într-un operand. În ambele cazuri registrul indicator de stivă (SP) se modifică corespunzător (prin decrementare și respectiv incrementare) astfel încât registrul SP să indice poziţia curentă a vârfului de stivă. Sintaxa instrucțiunilor este:

```
PUSH <parametru_1>
POP <parametru_1>
```

Transferul se face numai pe 16 biţi. Aceste instrucțiuni sunt utile pentru salvarea temporară şi refacerea conținutului unor registre. Aceste operații sunt necesare mai ales la apelul de rutine și la revenirea din rutine.

Exemple:

```
push ax          push var16
pop bx           pop var16
```

Directive și instrucțiuni ASM
Instrucţiuni aritmetice

Aceste instrucţiuni efectuează cele patru operații aritmetice de bază: adunare, scădere, înmulţire și împărţire. Rezultatul acestor instrucțiuni afectează starea indicatorilor de condiţie.

Instrucţiunile ADD și ADC

Aceste instrucţiuni efectuează operaţia de adunare a doi operanzi, rezultatul plasându-se în primul operand. A două instrucțiune ADC (ADD with carry) în plus adună și conținutul indicatorului de transport CF. Această instrucţiune este utilă pentru implementarea unor adunări în care operanzii sunt mai lungi de 32 de biţi.

```
ADD <parametru_1>,<parametru_2>
ADC <parametru_1>,<parametru_2>
```

Exemple:

```
add ax, 1234h
add bx, ax
adc dx, var16
```

Directive și instrucțiuni ASM
Instrucţiuni aritmetice (cont.)

Instrucţiunile SUB și SBB

Aceste instrucțiuni implementează operaţia de scădere. A două instrucţiune, SBB (Subtract with borrow) scade și conținutul indicatorului CF, folosit în acest caz pe post de bit de împrumut. Ca şi ADC, SBB se foloseşte pentru operanzi de lungime mai mare.

```
SUB    <parametru_1>,<parametru_2>
```

Instrucţiunile MUL și IMUL

Aceste instrucțiuni efectuează operaţia de înmulţire, MUL pentru întregi fără semn şi IMUL pentru întregi cu semn. De remarcat că la operațiile de înmulţire și împărţire trebuie să se tină cont de forma de reprezentare a numerelor (cu semn sau fără semn), pe când la adunare și scădere acest lucru nu este necesar. Pentru a evita dese depăşiri de capacitate s-a decis ca rezultatul operaţiei de înmulţire să se păstreze pe o lungime dublă faţă de lungimea operanzilor. Astfel dacă operanzii sunt pe octet rezultatul este pe cuvânt, iar dacă operanzi sunt pe cuvânt rezultatul este pe dublu-cuvânt. De asemenea se impune ca primul operand şi implicit şi rezultatul să se păstreze în registrul acumulator. De aceea primul operand nu se mai specifică.

```
MUL <parametru_2>
IMUL <parametru_2>
```

Directive și instrucțiuni ASM
Instrucţiuni aritmetice (cont.)

Instrucţiunile MUL și IMUL (cont.)

Exemple:

```
mul dh          ; AX<=AL*DH
mul bx          ; DX:AX<= AX*BX    DX este extensia registrului acumulator AX
imul var8       ; AX<=AL*var8
```

Instrucţiunile DIV și IDIV

Aceste instrucțiuni efectuează operaţia de împărţire pe întregi fără sem și respectiv cu semn. Pentru a creşte plaja de operare se consideră că primul operand, care în mod obligatoriu trebuie să fie în acumulator, are o lungime dublă faţă de al doilea operand. Primul operand nu se specifică.

```
DIV <parametru_2>
IDIV <parametru_2>
```

Exemple:

```
div  cl         ;AL<=AX/CL și AH<=AX modulo CL (adică restul împărţirii)
div  bx         ;AX<= (DX:AX)/BX și DX<=(DX:AX) modulo BX
```

Directive și instrucțiuni ASM
Instrucţiuni aritmetice (cont.)

Instrucţiunile INC și DEC

Aceste instrucțiuni realizează incrementarea și respectiv decrementarea cu o unitate a operandului. Aceste instrucţiuni sunt eficiente ca lungime și ca viteză. Ele se folosesc pentru contorizare și pentru parcurgerea unor şiruri prin incrementarea sau decrementarea adreselor.

```
INC <parametru>
DEC <parametru>
```

Exemple:

```
inc   și            ; SI<=SI+1
dec  cx             ; CX<=CX-1
```

Instrucţiunea CMP

Această instrucțiune compară cei doi operanzi prin scăderea lor. Dar rezultatul nu se memorează; instrucţiunea are efect numai asupra indicatorilor de condiţie. Această instrucțiune precede de obicei o instrucțiune de salt condiţionat. printr-o combinație de instrucțiune de comparare și o instrucțiune de salt se pot verifica relaţii de egalitate, mai mare, mai mic, mai mare sau egal, etc.

CMP <parametru_1>, <parametru_2>

Exemplu:

```
cmp ax, 50h
```

Directive și instrucțiuni ASM
Instrucţiuni logice

Aceste instrucțiuni implementează operațiile de bază ale logicii booleene. Operaţiile logice se efectuează la nivel de bit, adică se combină printr-o operaţie logică fiecare bit al operandului 1 cu bitul corespunzător din operandul al doilea. Rezultatul se generează în primul operand.

Instrucţiunile AND, OR, NOT şi XOR

Aceste instrucțiuni implementează cele patru operații de bază: ŞI, SAU, Negaţie şi SAU-Exclusiv.

```
AND <parametru_1>,<parametru_2>
OR <parametru_1>, <parametru_2>
NOT <parametru_1>
XOR <parametru_1>,<parametru_2>
```

Exemple:

```
and   al, 0fh
or    bx, 0000111100001111b
and   al,ch
xor   ax,ax          ; şterge conținutul lui ax
```

Directive și instrucțiuni ASM
Instrucţiuni logice (cont.)

Instrucţiunea TEST

Această instrucţiune efectuează operaţia ŞI logic fără a memora rezultatul. Scopul operaţiei este de a modifica indicatorii de condiţie. Instrucţiunea evită distrugerea conținutului primului operand.

```
TEST <parametru_1>,<parametru_2>
```

Exemple:

```
      test al, 00010000b  ;se verifică dacă bitul D4 din al
este zero sau nu
      test bl, 0fh        ;se verifica dacă prima cifră
hexazecimală din bl este 0
```

CAP VII

Nivelul sistemului de exploatare

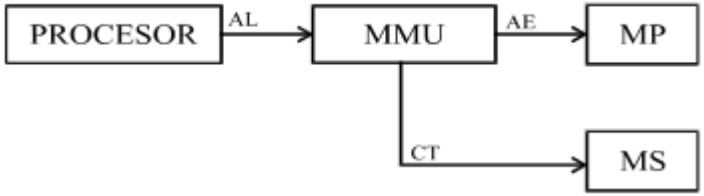
Memoria virtuală, conceptual de paginare , conceptul de segmentare

Memoria virtuală reprezintă o tehnică de organizare a memoriei prin intermediul căreia programatorul accesează un spaţiu virtual de adresare foarte mare. Tehnica folosită este maparea în memoria fizică disponibilă.

Uzual, spaţiul virtual de adrese corespunde discului magnetic, pentru programator fiind transparente mecanismele de memorie virtuală (MV), având posibilitatea sa adreseze o memorie (virtuală) de capacitatea hard-discului și nu de capacitatea memoriei fizice preponderentă DRAM.

În cazul MV, memoria principală este analoagă memoriei cache situată între CPU (*Central Processing Unit*) și memoria principală, numai că de această dată ea se situează între CPU și discul hard. Deci memoria principală (MP) se comportă oarecum ca un cache între CPU și discul hard. Prin mecanismele de MV se măreşte probabilitatea ca informaţia să fie accesată (de către CPU) din spaţiul virtual (HD).

Pentru implementarea conceptului de memorie virtuală se foloseşte următoarea schemă de principiu:



AL reprezintă adresa logică care este generată de CPU prin citirea instrucţiunii curente. Prin prelucrarea adresei logice în MMU (hardware) rezultă adresa fizică, externă şi memoria principală.

Memoria virtuală, conceptual de paginare , conceptul de segmentare

MMU sau PMMU (Page Memory Management Unit) realizează o funcție de translatare:

$$fT^*: AL \rightarrow AE.$$

fT realizează translatarea între adresele logice (virtuale) şi cele fizice (reale). Dacă adresa logică nu are corespondent fizic în memoria principală, se generează comenzile de transfer (CT) între memoria externă şi memoria principală.

Realizarea practică a conceptului de memorie virtuală se bazează pe transferul de blocuri între memoria externă şi memoria principală. Cea mai răspândită metodă de implementare este metoda segmentării memoriei.

S-a arătat că fiecare instrucţiune la nivel cod maşină este formată din 2 câmpuri mari:

OPCODE	ADRESĂ
n	l
L	

În câmpul de adresă apar elementele constitutive ale adresei logice din care, prin efectuarea unor calcule, se determină adresa efectivă sau adresa fizică.

În cazul adresei efective:

AE=f(AL)

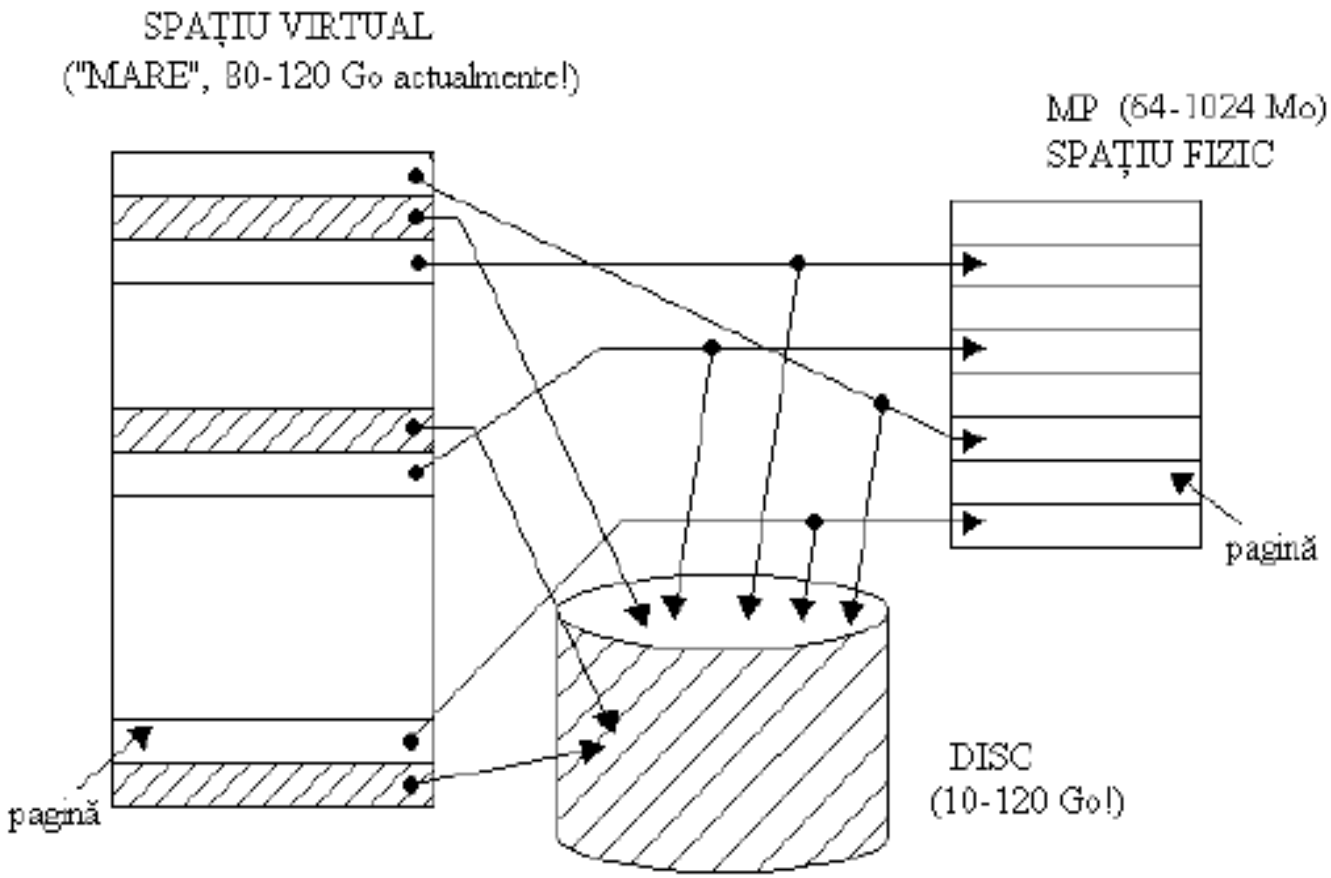
Unde

AL: adresa logică;

AE: adresa efectivă.

Memoria virtuală, conceptual de paginare , conceptul de segmentare

De obicei, spaţiul virtual de adresare este împărţit în entităţi de capacitate fixă (4 ¸ 64 Ko actualmente), numite pagini. O pagină poate fi mapată în MP sau pe disc.



Paginarea

O pagină de memorie sau o pagină virtuală este un bloc contiguu de memorie virtuală cu lungime fixă, descris printr-o singură intrare într-un tabelul de paginare.

Aceasta este cea mai mică unitate de date pentru gestionarea memoriei într-un sistem de operare pentru memoria virtuală.

În mod similar, un cadru de pagină (page frame) este cel mai mic bloc de memorie fizică cu lungime fixă în care sunt mapate paginile de memorie de către sistemul de operare.

Un transfer de pagini între memoria principală și cea auxiliară, cum ar fi un hard disk, este denumit paginare (sau swapping).

Paginarea (cont.)

În general, prin mecanismele de MV, MP conține paginile cel mai recent accesate de către un program, ea fiind după cum am mai arătat, pe post de “cache” între CPU și discul hard.

Transformarea adresei virtuale emisă de către CPU într-o adresă fizică (existentă în spaţiul MP) se numește mapare sau translatare. Aşadar mecanismul de MV oferă o funcție de relocare a programelor (adreselor de program), pentru că adresele virtuale utilizate de un program sunt relocate spre adrese fizice diferite, înainte ca ele să fie folosite pentru accesarea memoriei.

Această mapare permite aceluiaşi program să fie încărcat și să ruleze oriunde în MP, modificările de adrese realizându-se automat prin mapare (fără MV un program depinde de obicei în execuția sa de adresa de memorie unde este încărcat).

MV este un concept deosebit de util în special în cadrul sistemelor de calcul multiprogramate care - de exemplu prin “*time-sharing*” – permit execuția cvasi-simultană a mai multor programe (vezi sistemul de operare WINDOWS XX, Unix, Ultrix etc.). Fiecare dintre aceste programe are propriul său spaţiu virtual de cod și date (având alocate un număr de pagini virtuale), dar în realitate toate aceste programe vor partaja aceeaşi MP, care va conține dinamic, pagini aferente diverselor procese.

Paginarea (cont.)

În implementarea MV trebuie avute în vedere următoarele aspecte importante:

- paginile să fie suficient de mari (4 ko ¸16 ko … 64 ko) astfel încât să compenseze timpul mare de acces la disc (9 ¸12 ms).
- organizări care să reducă rata de evenimente PF, rezultând un plasament flexibil al paginilor în memorie (MP)
- PF-urile trebuie tratate prin software şi nu prin hardware (spre deosebire de miss-urile în cache-uri), timpul de acces al discurilor permitând lejer acest lucru.
- scrierea în MV se face după algoritmi tip “Write Back” și nu “Write Through” (ar consuma timp enorm).

Paginarea (cont.)

- Fiecare program are propria sa tabelă de pagini care mapează spaţiul virtual de adresare al programului într-un spaţiu fizic, situat în MP.
- Tabela de pagini + PC + registrele microprocesorului formează starea unui anumit program.
- Programul + starea asociată caracterizează un anumit proces (task).
- Un proces executat curent este activ, altfel el este inactiv.
- Comutarea de taskuri implică inactivarea procesului curent şi activarea altui proces, inactiv până acum rezultând ca fiind necesară salvarea/restaurarea stării proceselor.
- Sistemul de operare (S.O.) trebuie doar să reâncarce registrul pointer al adresei de bază a paginii (PTR) pentru a pointa la tabela de pagini aferentă noului proces activ.

Memoria virtuală, conceptual de paginare , conceptul de segmentare

Segmentarea

Aceasta constituie o altă variantă de implementare a MV, care utilizează în locul paginilor de lungime fixă, entităţi de lungime variabilă numite segmente.

În segmentare, adresa virtuală este constituită din 2 cuvinte: o bază a segmentului și respectiv un deplasament (offset) în cadrul segmentului.

Datorită lungimii variabile a segmentului (de ex. 1 octet ¸ 2^{32} octeți la arhitecturile Intel Pentium), trebuie făcută și o verificare a faptului că adresa virtuală rezultată (baza + offset) se încadrează în lungimea adoptată a segmentului.

Segmentarea oferă posibilităţi de protecţie puternice și sofisticate a segmentelor.

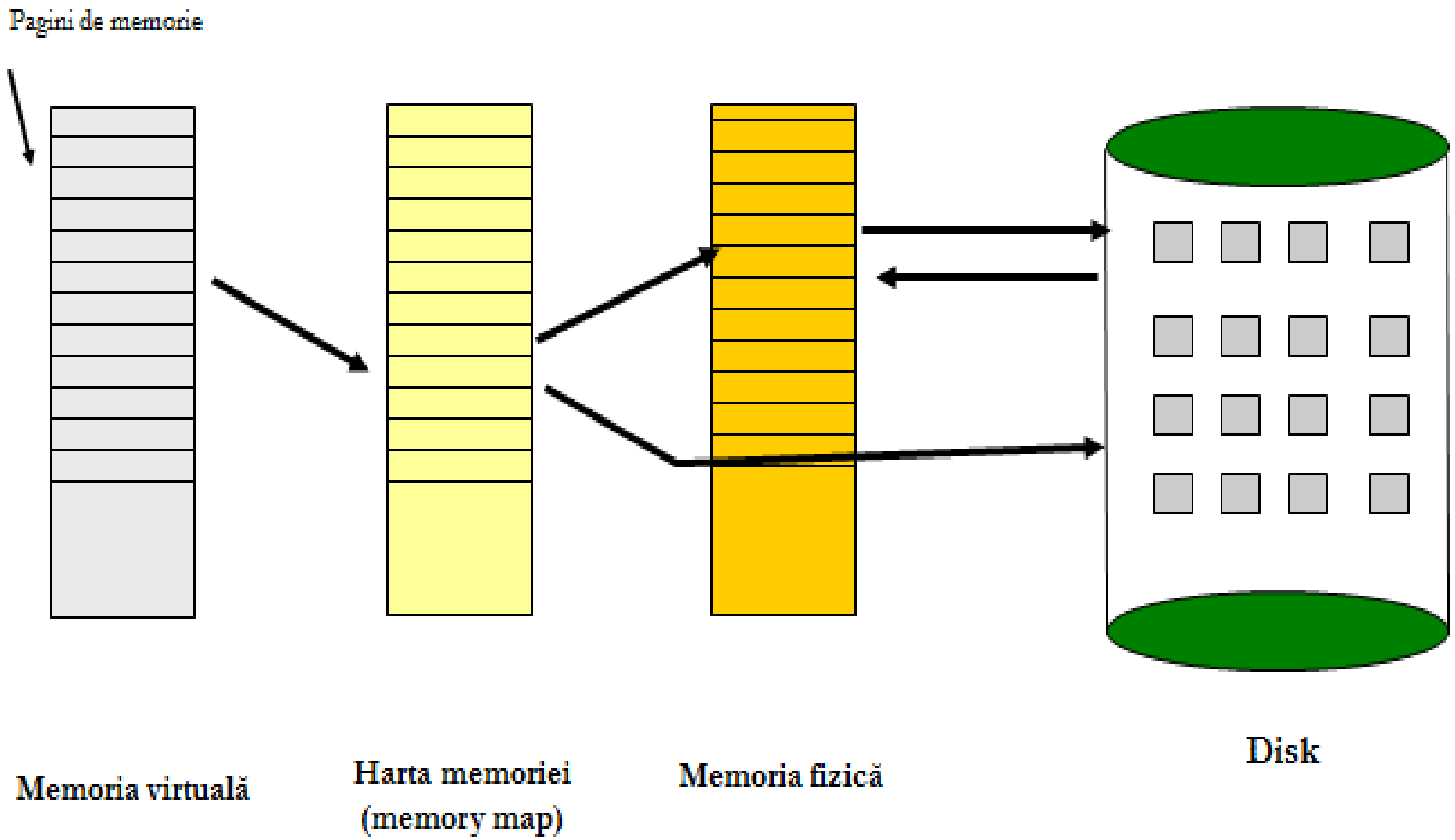
Pe de altă parte, segmentarea induce şi numeroase dezavantaje precum - 2 cuvinte necesare pentru o adresă virtuală, având în vedere lungimea variabilă a segmentului. Asta complică sarcina compilatoarelor și a programelor:

- încărcarea segmentelor variabile în memorie este mai dificilă decât la paginare
- fragmentarea memoriei principale (exista porţiuni nefolosite)
- frecvent, trafic ineficient MP-disc (de exemplu pentru segmente “mici”, transferul cu discul fiind complet ineficient – accese la nivel de sector = 512 octeți)
- există în practică și implementări hibride segmentare – paginare

Memoria virtuală, conceptual de paginare , conceptul de segmentare

Memoria virtuală

Reprezintă separarea conceptuală a memoriei logice disponibile pentru aplicatii fata de memoria fizica. În acest mod putem avea o memorie virtuala de dimensiuni mari chiar cu o memorie fizica de dimensiuni reduse :



Memoria virtuală, conceptual de paginare , conceptul de segmentare

Memoria virtuală (cont.)

- Nu toate obiectele (date sau instrucțiuni) pot fi la un moment dat în memoria principală. Dacă avem memorie virtuală, atunci unele dintre obiecte se pot afla pe disc. Spaţiul de adresare este de regulă împărţit în blocuri de lungime fixă – pagini.
- La un moment dat, paginile se află fie în memoria principală, fie pe disc
- Atunci când se cere un obiect care nu este în cache sau în memoria principală, apare un “page-fault” – moment în care întreaga pagină este mutată de pe disc în memoria principală. Aceste “page-fault” durează mai mult şi atunci sunt controlate de software şi UCP nu face pauză.
- De regulă, UCP comută către alt task atunci când apare un acces la disc. Memoria cache şi memoria principală au aceeaşi relaţie ca și cea existentă între memoria principală și disc.
- În orice moment, un calculator rulează mai multe procese, fiecare având propriul spaţiu de adrese de memorie. Ar fi foarte costisitor să se dedice un întreg spaţiu de adresare pentru fiecare proces, având în vedere că multe dintre procese folosesc doar o mică parte a spaţiului propriu de adrese. A apărut astfel necesitatea partajării unei părţi a memoriei între mai multe procese.

Memoria virtuală (cont.)

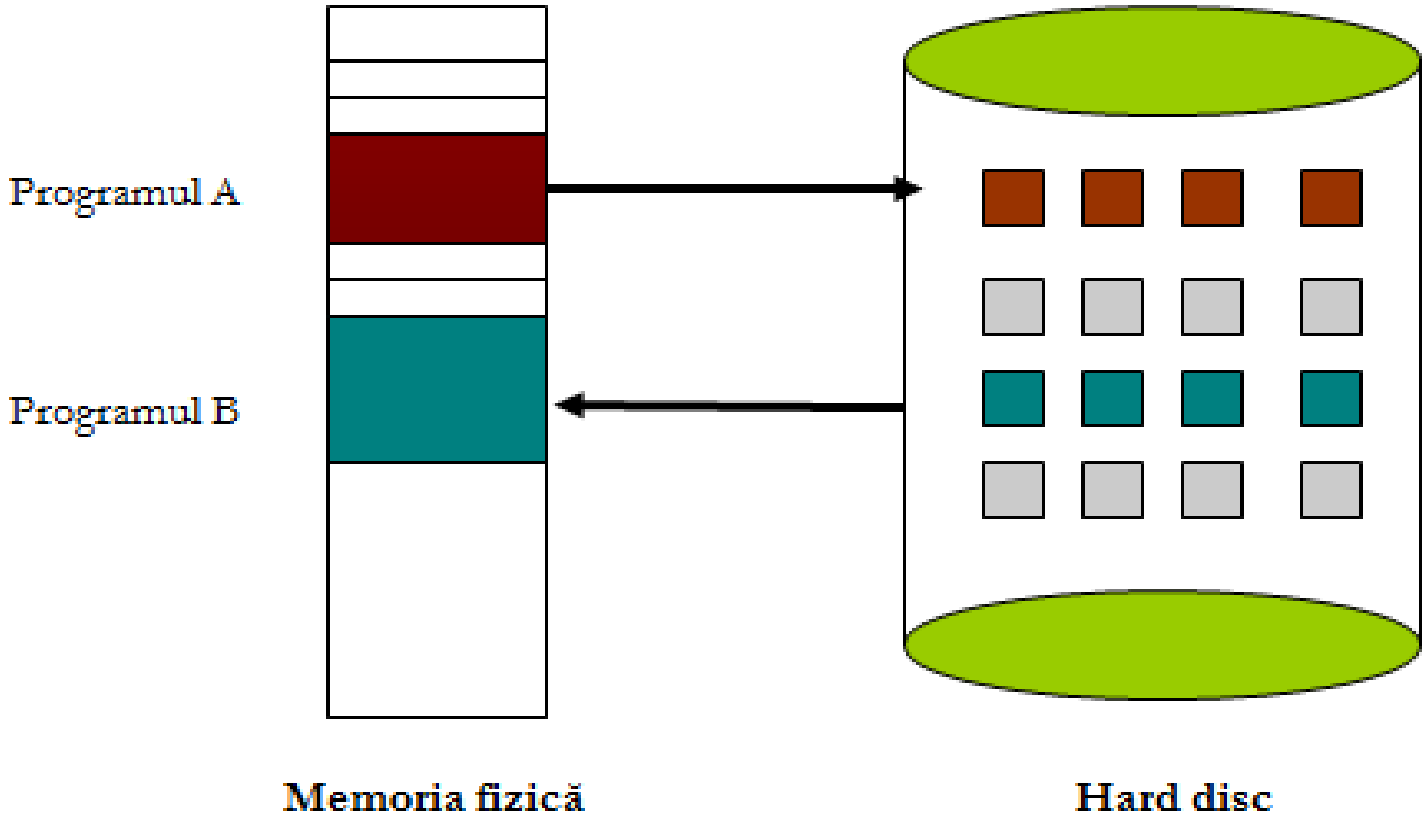
- Acest procedeu poartă numele de “memorie virtuală” – memoria fizică se divide în blocuri care sunt alocate diferitelor procese.
- Inerentă unei astfel de abordări este o schemă de protecţie ce restricţionează accesul proceselor la blocuri ce aparţin altor procese. Majoritatea formelor de memorie virtuală reduc, de asemenea, timpul de pornire a unui program, deoarece nu tot codul sau datele trebuie să fie deja în memoria fizică înainte ca programul să înceapă.
- Însă partajarea între procese a memoriei nu este adevăratul motiv pentru care s-a inventat memoria virtuală. Dacă un program devine prea mare pentru memoria fizică, este sarcina programatorului să îl facă să încapă în ea. Au rezultate acele suprapuneri (overlay).
- Blocurile de memorie în cazul memoriei virtuale se numesc *pagini* sau *segmente*. UCP foloseşte adrese virtuale ce sunt translatate (hardware cât și software) în adrese fizice ce accesează memoria principală. Acest procedeu se numește procedeul de mapare a memoriei sau de translatare a adreselor. Astăzi memoria virtuală intervine la nivel de memorie principală și disc magnetic

Memoria virtuală, conceptual de paginare , conceptul de segmentare

Memoria virtuală (cont.)

Cererea de pagini (*demand paging*)

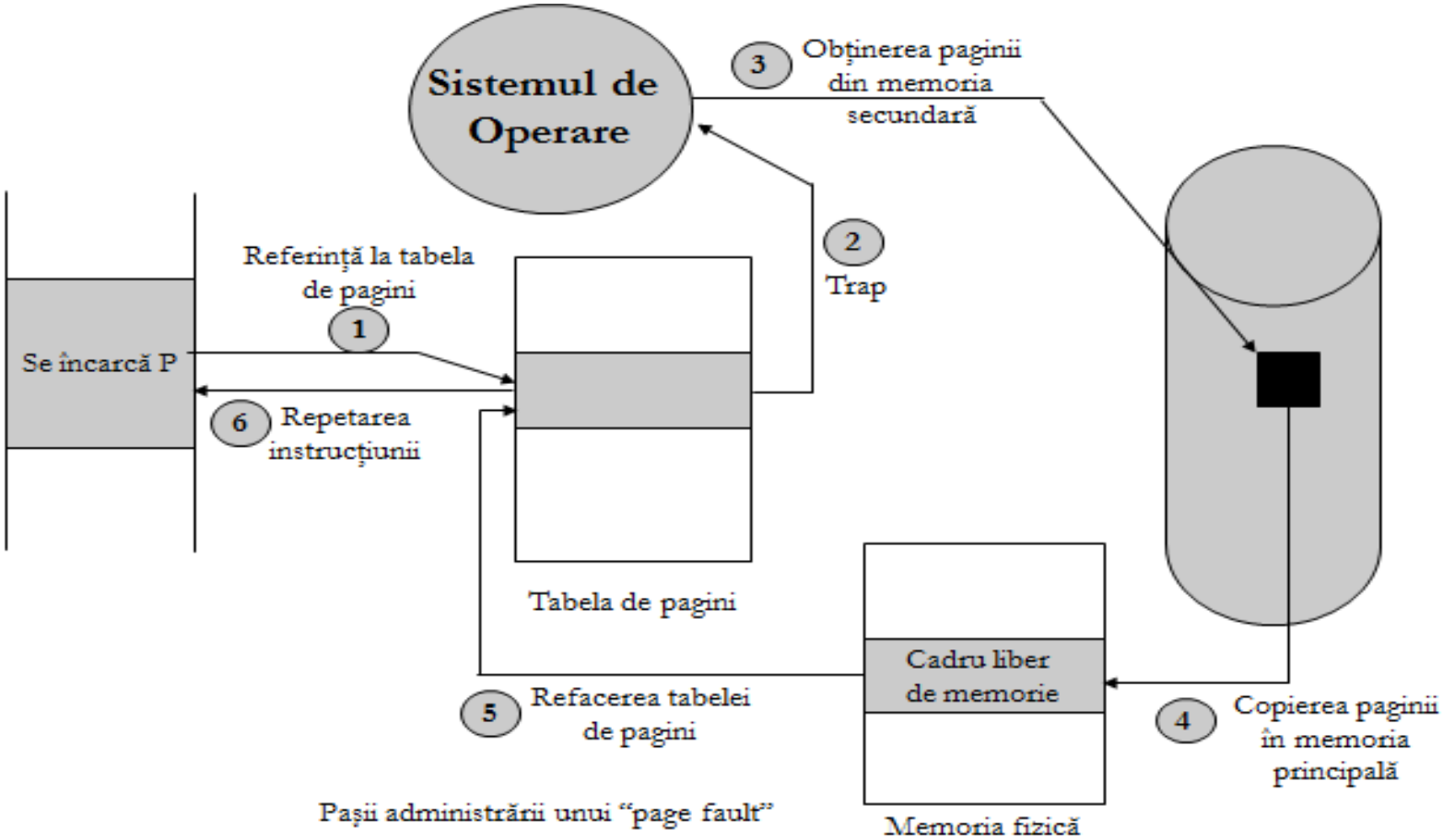
Atunci când o pagină de memorie este referită (fie că este vorba despre cod sau date) și ea nu se află în memorie atunci ea este adusă de pe disc şi se re-execută instrucţiunea



Memoria virtuală, conceptual de paginare , conceptul de segmentare

Memoria virtuală (cont.) **Cererea de pagini** (*demand paging*)

Paşii ce se urmăresc în cazul unui “page fault”:



Paşii administrării unui “page fault”

- În cazul supra-alocării memoriei trebuie să renunţăm la ceva deja existent în memorie.
- Supra-alocarea apare atunci când programele au nevoie de mai multe pagini de memorie decât cele existente fizic.
- Metoda de abordare: dacă nici o pagină fizică nu este liberă, se caută una care nu este utilizată la momentul respectiv şi se eliberează.